



CIBERSEGURIDAD EN ENTORNOS DE LAS TECNOLOGÍAS DE LA INFORMACIÓN

Puesta en Producción Segura

TEMA 1

Prueba de aplicaciones web y para dispositivos móviles
Alberto Diéguez Álvarez

Apartado 1: Configurar el entorno	3
Instalar PHP junto con la librería PHPUnit	3
Instalar Visual Studio Code y configurar el entorno para poder trabajar con PHP y poder realizar las pruebas unitarias en el mismo entorno.....	4
Apartado 2: Rellenar la siguiente tabla utilizando el código del programa de pruebas que se ha dado como ejemplo (ejemploTest.php).....	5
Apartado 3: Crear un programa PHP que genere la secuencia de Fibonacci. ...	7
Crearemos el archivo fibo.php	7
Debes escribir un código donde incluirás:.....	7
Dentro de este archivo crearemos una función llamada <i>fibonacci</i> que devolverá la secuencia de fibonacci hasta el valor indicado como parámetro. Por ejemplo, fibonacci(6) devolverá [0,1,1,2,3,5] y fibonacci(8) devolverá [0,1,1,2,3,5,8]	7
Apartado 4: Creación del programa de pruebas unitarias	8
❖ Crearemos un archivo de nombre testFibo.php.....	8
❖ Importaremos la librería de testeo de software en este caso PHPUnit. ...	8
❖ Importaremos el archivo donde se ha definido la función Fibonacci	8
❖ Crearemos nuestra clase (del tipo TestCase)	8
❖ Dentro de esta clase definiremos una función (es recomendable un nombre ilustrativo que empiece por test)	8
❖ Dentro de la función reflejaremos el tipo de testeo que vamos a realizar. En este caso nuestro objetivo es verificar si la posición X de la sucesión de Fibonacci coincide con el resultado esperado, es decir coincide con la posición X que devuelve nuestro programa. Es decir queremos verificar que el quinto número de la sucesión, que es 3, coincide con el quinto número que devuelve nuestra secuencia.....	8
Apartado 5: Verificación de software	9
Ejecutaremos el programa final de pruebas (testFibo.php) y verificaremos si realmente nuestro programa que calcula la sucesión de Fibonacci se comporta como esperamos.....	9
Si el programa que comprueba el código detecta un error nos reflejará que dato está esperando y que ha recibido	9
Apartado 6: Responder a la pregunta	10

¿Qué te pedimos que hagas?

Antes de nada, reflejaremos la sucesión de Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34... Tienes más información sobre esta interesante sucesión y su origen [aquí](#)

Apartado 1: Configurar el entorno

Para realizar la tarea los pasos son:

Instalar PHP junto con la librería PHPUnit

Entro a la web <https://windows.php.net/download/> y descargo el zip VS17 x64 Thread Safe (2025-Nov-19 10:04:28).

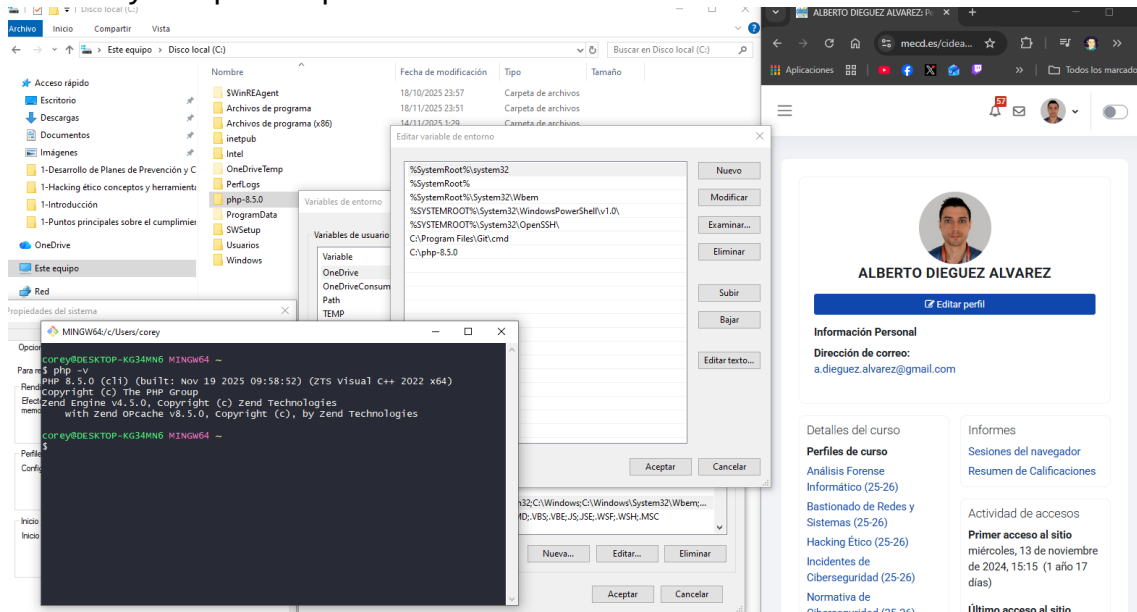
The screenshot shows the PHP Windows download page. The main content area lists two download options for PHP 8.5.0:

- VS17 64 Non Thread Safe (2025-Nov-19 10:02:04)**
 - Zip [32.96MB]
sha256: c4d3bfe570347b3b5f136e47d3ca16448e711e87f0eabdcdd1817d78327774f
 - Debug Pack [37.44MB]
sha256: d560778d852bb9c226192190565ace1161999278b38d8c9db90f585b36e45b0c
 - Development package (SDK to develop PHP extensions) [1.63MB]
sha256: c76ee00b826e3f02b566544ee697893cbed5d9f491d9e02557a87387081e9009
- VS17 x64 Thread Safe (2025-Nov-19 10:04:28)**
 - Zip [33.08MB]
sha256: 0f50648ed1a60c9ea108c5085c5715d977e1b4fddc7c8a6acd37db65646be5
 - Debug Pack [37.61MB]
sha256: 1711a70b5a1c2dc99d8ff01067fbbf972fb216fe903e0a13b4212f595238adb
 - Development package (SDK to develop PHP extensions) [1.64MB]
sha256: 67538e544b599b049f855c9b76609aa6a634443ae812d5d4d5ad61

The sidebar on the right shows a user profile for ALBERTO DIEGUEZ ALVAREZ, including a profile picture, a bio, and a list of courses and activities.

Instalo PHP:

Renombro la carpeta de php, la pego en C: , añado el path a las variables de sistema y compruebo por consola la instalación.

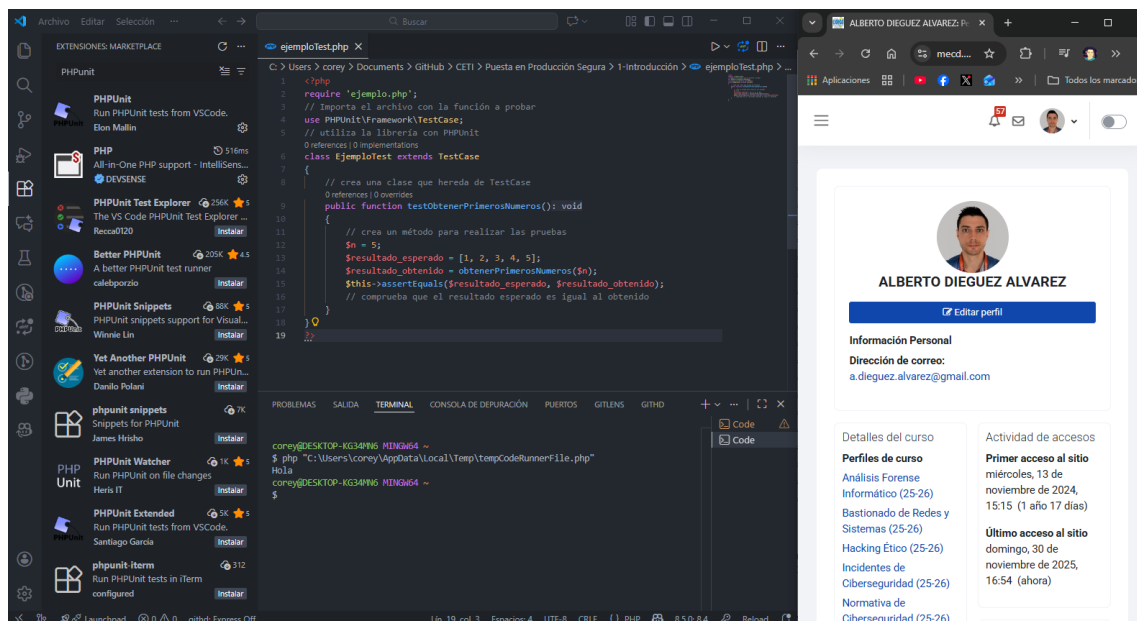


Instalar Visual Studio Code y configurar el entorno para poder trabajar con PHP y poder realizar las pruebas unitarias en el mismo entorno

Descargo VScode en la web: <https://code.visualstudio.com/download> .

Lo Instalo y añado las siguientes extensiones.

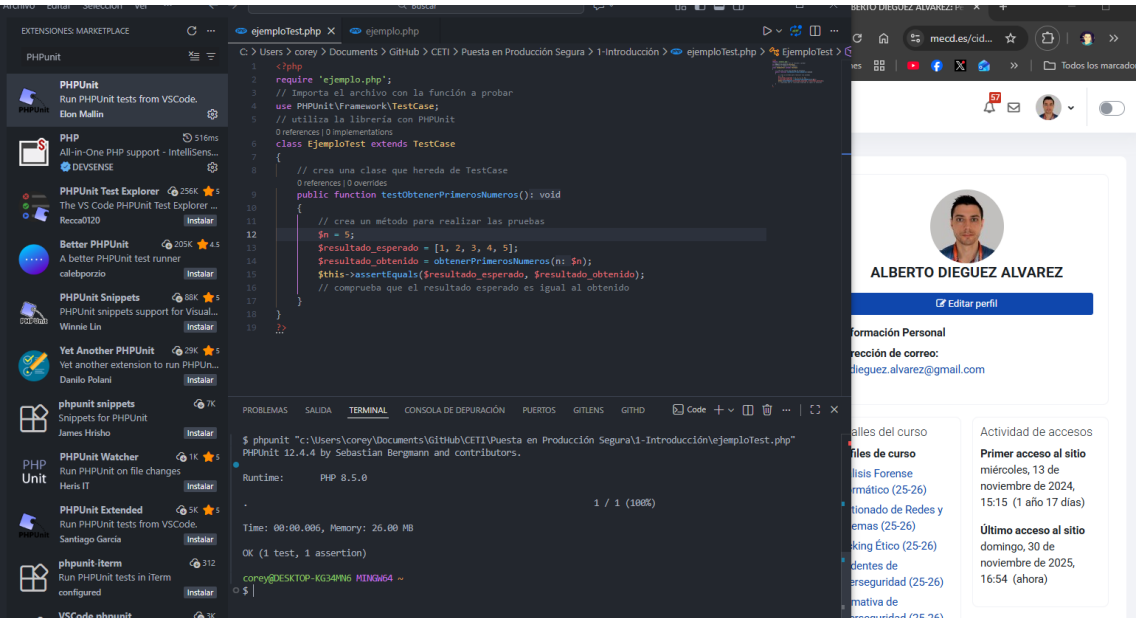
- **PHPUnit**



Instalo PHPUnit dese <https://phpunit.de/getting-started/phpunit-12.html>

Sigo los pasos y ajusto el php.ini para poder ejecutar el comando ´phpunit´.

Ejecuto el test de ejemplo:



Apartado 2: Rellenar la siguiente tabla utilizando el código del programa de pruebas que se ha dado como ejemplo (ejemploTest.php)

ejemploTest.php:

LINEA	CODIGO
1	<?php
2	require 'ejemplo.php';
3	// Importa el archivo con la función a probar
4	use PHPUnit\Framework\TestCase;
5	// utiliza la librería con PHPUnit
6	class EjemploTest extends TestCase {
7	// crea una clase que hereda de TestCase
8	public function testObtenerPrimerosNumeros() {
9	// crea un método para realizar las pruebas
10	\$n = 5;
11	\$resultado_esperado = [1, 2, 3, 4, 5];

12	\$resultado_obtenido = obtenerPrimerosNumeros(\$n);
13	\$this->assertEquals(\$resultado_esperado, \$resultado_obtenido);
14	// comprueba que el resultado esperado es igual al obtenido
15	}
16	}
17	?>

PREGUNTA	RESPUESTA
Indica el número de las líneas donde hay un comentario	3 // Importa el archivo con la función a probar 5 // utiliza la librería con PHPUnit 7 // crea una clase que hereda de TestCase 9 // crea un método para realizar las pruebas 14 // comprueba que el resultado esperado es igual al obtenido
Indica el número de las líneas donde hay comandos de pre-procesamiento	2 require 'ejemplo.php'; 4 use PHPUnit\Framework\TestCase;
Indica el nombre de las variables utilizadas	10 \$n = 5; 11 \$resultado_esperado = [1, 2, 3, 4, 5]; 12 \$resultado_obtenido = obtenerPrimerosNumeros(\$n); 13 \$this->assertEquals(\$resultado_esperado, \$resultado_obtenido);
Indica el nombre de los métodos/funciones utilizados	8 public function testObtenerPrimerosNumeros() {
Indica qué hace la línea 13	13 \$this->assertEquals(\$resultado_esperado, \$resultado_obtenido); Es la línea se aserción, donde comparamos los resultados, y si no son iguales el test falla.

Apartado 3: Crear un programa PHP que genere la secuencia de Fibonacci.

Se pide crear un pequeño programa PHP para generar la secuencia de Fibonacci. Se ha escogido PHP porque se va a utilizar en las unidades 2 y 3 y además, es uno de los lenguajes más utilizados en las aplicaciones web

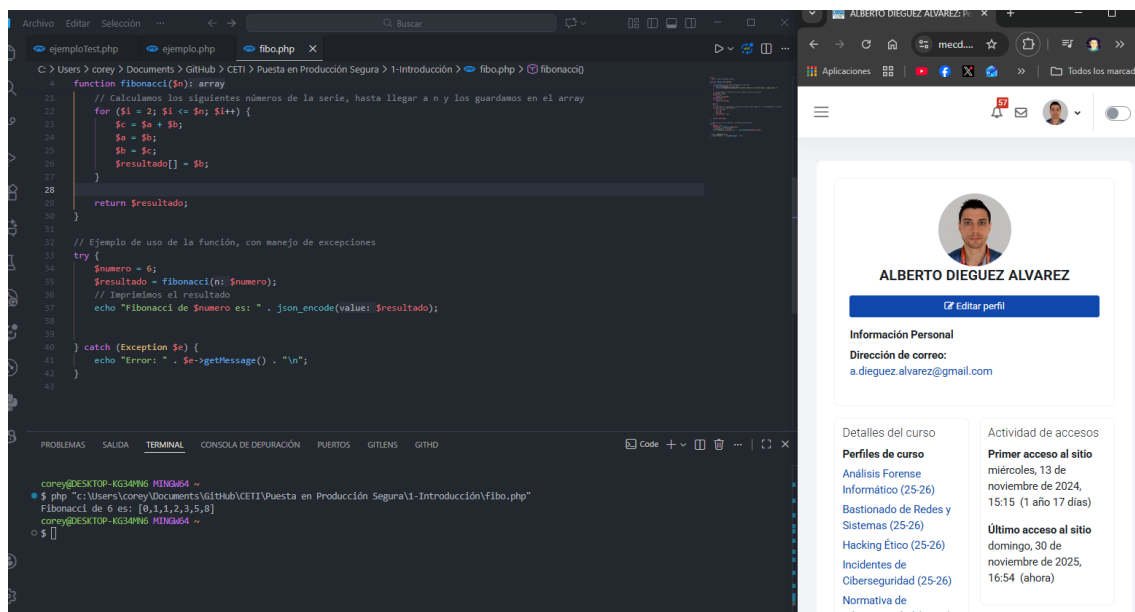
Para realizar la tarea los pasos serán:

Crearemos el archivo `fibo.php`

Debes escribir un código donde incluirás:

- Una línea de comentario donde escribirás tu nombre.
- Otros comentarios en las líneas que creas importantes
- Utilizar variables
- Utilizar alguna estructura de control
- Utilizar el control de errores (mira este enlace <https://www.php.net/manual/en/language.exceptions.php>).

Dentro de este archivo crearemos una función llamada *fibonacci* que devolverá la secuencia de fibonacci hasta el valor indicado como parámetro. Por ejemplo, `fibonacci(6)` devolverá `[0,1,1,2,3,5]` y `fibonacci(8)` devolverá `[0,1,1,2,3,5,8]`



The screenshot shows a code editor with a PHP file named `fibo.php`. The code defines a function `fibonacci($n): array` that calculates the Fibonacci sequence up to `$n` and returns it as an array. It includes comments in Spanish and uses a `try-catch` block to handle exceptions. The terminal shows the command `$ php "c:\Users\corey\Documents\Github\CETI\Puesta en Producción Segura\1-Introducción\fibo.php"` and the output `Fibonacci de 6 es: [0,1,1,2,3,5,8]`. To the right, a browser window shows a user profile for **ALBERTO DIEGUEZ ALVAREZ** with a profile picture, a bio, and a list of course profiles including `Análisis Forense Informático`, `Bastionado de Redes y Sistemas`, `Hacking Ético`, `Incidentes de Ciberseguridad`, and `Normativa de Ciberseguridad`.

Apartado 4: Creación del programa de pruebas unitarias

Crearemos un programa principal donde definiremos una clase llamada Test para probar nuestro código (tienes un ejemplo en el enunciado de la tarea)

Para realizar la tarea los pasos serán:

- ❖ Crearemos un archivo de nombre testFibo.php
- ❖ Importaremos la librería de testeo de software en este caso PHPUnit.
- ❖ Importaremos el archivo donde se ha definido la función Fibonacci
- ❖ Crearemos nuestra clase (del tipo TestCase)
- ❖ Dentro de esta clase definiremos una función (es recomendable un nombre ilustrativo que empiece por test)
- ❖ Dentro de la función reflejaremos el tipo de testeo que vamos a realizar. En este caso nuestro objetivo es verificar si la posición X de la sucesión de Fibonacci coincide con el resultado esperado, es decir coincide con la posición X que devuelve nuestro programa. Es decir queremos verificar que el quinto número de la sucesión, que es 3, coincide con el quinto número que devuelve nuestra secuencia.
- Para hacer esta comprobación usaremos el tipo de comprobación **assertEquals** que comprueba si dos valores son iguales
- Mediante esta función comprobaremos si la posición quinta de nuestra función coincide con el valor esperado (el valor esperado es la quinta posición de la sucesión que es 3)

The screenshot displays a code editor with a file named `testFibo.php`. The code uses PHPUnit to test a `Fibonacci` function. It defines a `TestFibo` class extending `TestCase` and includes a `testFibonacciPosicion5()` method. This method sets an expected result of 3 for the 5th position, obtains the actual result from the `Fibonacci` function, and uses `$this->assertEquals()` to verify they match. The terminal at the bottom shows the command `$ phpunit "C:\Users\corey\Documents\GitHub\CETI\Puesta en Producción Segura\1-Introducción\testFibo.php"` being executed, resulting in `OK (1 test, 1 assertion)`. To the right, a browser window shows a user profile for **ALBERTO DIEGUEZ ALVAREZ** with contact information and course details.

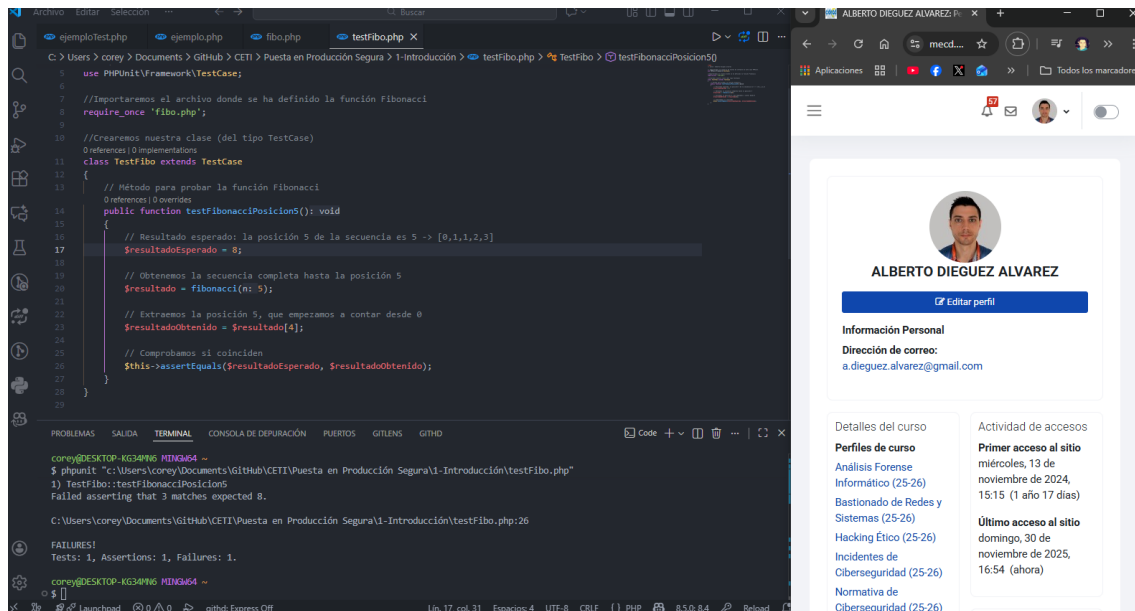
Apartado 5: Verificación de software

Ejecutamos la prueba y comprobamos que no hay error

Pasos a realizar:

Ejecutaremos el programa final de pruebas (testFibo.php) y verificaremos si realmente nuestro programa que calcula la sucesión de Fibonacci se comporta como esperamos.

Si el programa que comprueba el código detecta un error nos reflejará que dato está esperando y que ha recibido



Apartado 6: Responder a la pregunta

Dado el siguiente código PHP que comprueba la conexión a una base de datos MySQL y que si se ejecuta funciona:

```
1 <?php
2 require_once 'pdoconfig.php';
3 $servername = "db1";
4 $database = "databasename";
5 $username = "username";
6 $password = "password";
7
8 try {
9     $conn = new PDO("mysql:host=$servername;dbname=$database",
10 $username, $password);
11     echo "Conectado a $database de $servername con exito.";
12 } catch (PDOException $pe) {
13     die("No es posible la conexión a $database :". $pe->getMessage());
14 }
15 ?>
```

Si realizamos un análisis estático de código nos va a devolver un error del tipo **"Hard-coded credentials are security-sensitive" ¿Por qué? Razona la respuesta. Indica la(s) línea(s) que habría que cambiar**

El error es debido a que escribir directamente credenciales o datos sensibles hardcoded es una muy mala práctica, ya que esos datos por lo que sea, se pueden subir a git... o pueden quedar fácilmente accesibles por cualquier tercero que simplemente con revisar el código tendría toda esa información.

Para solucionarlo, lo que haría sería **cargar las variables desde un archivo de configuración externo** que no se suba a git ni sea accesible desde fuera del servidor.

Por ejemplo, crear un archivo llamado config.php y que tenga los datos de las variables.

```
<?php
return [
    'host' => 'db1',
    'dbname' => 'databasename',
    'user' => 'username',
    'pass' => 'password'
];
?>
```

Ahora simplemente en nuestro archivo principal lo cargamos desde un path seguro.

```
$config = require '/path_seguro/config.php';
```

Ahora simplemente llamamos a las variables desde la variable \$config:

```
$conn = new PDO(
    "mysql:host={$config['host']};dbname={$config['dbname']}",
    $config['user'],
    $config['pass']
);
```

Con esta estructura nos aseguramos que el archivo con nuestros datos sensibles no sea accesible desde fuera.